AI / ML Practical Viva Cheat Sheet

Short theory • Code logic • Important viva answers • TY B.Sc. Cyber Security

Practical 1 — Anaconda + Basic Python

Aim: Install Anaconda and run a basic Python program.

Theory: Anaconda is a Python distribution for data science and machine learning. It includes Python, Jupyter Notebook, libraries and environment management.

```
print("Hello World")
```

Logic: print() displays output; the text inside quotes is a string.

Viva: Anaconda = Python/data-science distribution. Jupyter Notebook = interactive environment for executing code cell by cell. conda = package and environment manager.

Practical 2 — Decision Tree Malware Detection

Aim: Classify files as malware or benign using a Decision Tree.

Features: file_size, entropy, suspicious_imports, sections. Label: 0 = benign, 1 = malware.

Theory: A Decision Tree recursively splits data using conditions to create increasingly pure groups.

```
X = df.drop('label', axis=1)
y = df['label']
train_test_split(X, y, test_size=0.25)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Logic: X contains input features; y contains the target. Data is split into training/testing. fit() learns patterns; predict() predicts unseen data; accuracy compares predictions with actual labels.

Why entropy? Packed/encrypted malware can have high entropy because the data appears more random.

Viva: fit = train; predict = classify; label 1 = malware.

Practical 3/4 — Network Intrusion Detection

Aim: Detect suspicious network traffic using Decision Tree and Isolation Forest.

Features: duration, bytes_sent, connections. Label: 0 = normal, 1 = attack.

Decision Tree

Supervised learning because labelled examples are available. max_depth=3 limits tree complexity and helps reduce overfitting.

Isolation Forest

Unsupervised anomaly detection. It does not require labels during training. Unusual observations are isolated using fewer random splits.

Isolation Forest output: 1 = normal, -1 = anomaly (often converted to the practical's 0/1 format).

Key difference: Decision Tree = supervised classification; Isolation Forest = unsupervised anomaly detection.

stratify=y: preserves approximately the same class distribution in train and test sets.

Practical 5 — Facial Recognition Using PCA + KNN

Aim: Perform facial recognition/authentication using the Olivetti Faces dataset, PCA and KNN.

Dataset: 400 images, 40 people, 64×64 pixels = 4096 pixel features per image.

PCA

Principal Component Analysis reduces dimensionality while retaining important variation. It makes computation faster and reduces redundant information.

KNN

K-Nearest Neighbors identifies a face by finding the most similar enrolled faces. With K=1, the nearest single sample determines the class.

Authentication: distance ≤ threshold → ACCEPT; distance > threshold → REJECT.

FAR: False Acceptance Rate — unauthorized user accepted. **FRR:** False Rejection Rate — authorized user rejected.

Threshold trade-off: Higher threshold generally lowers FRR but increases FAR; lower threshold generally does the opposite.

Practical 6 — KNN Typing Authentication

Aim: Authenticate a user from typing behaviour. This is behavioural biometrics.

Features: typing_speed, key_hold_time, key_gap_time. Label: 1 = genuine, 0 = impostor.

Logic: A new typing pattern is compared with nearby training samples; the majority class among the K nearest samples becomes the prediction.

Why StandardScaler? KNN uses distance. Scaling prevents a feature with a larger numerical range from dominating.

```
scaler.fit_transform(X_train)
scaler.transform(X_test)
```

Important: The scaler learns parameters only from training data and applies those same parameters to test data.

Practical 7 — Fraud Detection

Aim: Detect fraudulent transactions using Logistic Regression and Random Forest.

Typical features: amount, time, location_risk, device_risk, num_transactions, balance_change.

Logistic Regression

A classification algorithm that estimates the probability of belonging to a class. Example: P(fraud)=0.91; with a 0.5 threshold, it is classified as fraud.

Random Forest

An ensemble of many Decision Trees. Each tree votes and the final class is generally selected by majority vote.

Class imbalance: If 95% transactions are normal and 5% are fraud, a model predicting everything as normal can still get about 95% accuracy while detecting no fraud. Therefore precision, recall and F1 are important.

Practical 8 — GAN Adversarial Attack

Aim: Demonstrate adversarial machine learning by generating modified fraud samples that may fool a fraud detector.

GAN: Generative Adversarial Network with a Generator and Discriminator.

```
Fraud sample + random noise → Generator → adversarial sample → Discriminator
```

Generator: creates modified samples. **Discriminator:** tries to distinguish generated samples from normal/real samples, depending on the implementation.

Adversarial example: an input deliberately modified so a model makes an incorrect prediction.

EPS: limits the size of perturbations so the generated change remains controlled.

BCELoss: Binary Cross-Entropy Loss for binary classification. **Adam:** optimizer used to update neural-network weights.

Practical 9 — Feature Engineering

Aim: Create more informative features for fraud detection.

```
amount_ratio = amount / avg_amount
is_night = (hour < 6).astype(int)
log_amount = np.log1p(amount)
distance_x_txn = distance_km * txn_last_24h
```

Amount ratio: compares current amount with usual amount. **is_night:** converts time into a binary feature. **log1p:** log(1+x), useful for reducing skew. **distance_x_txn:** combines two signals.

Feature engineering: creating better input variables from existing data so the model can identify useful patterns more effectively.

Practical 10 — ROC Curve and AUC

Aim: Evaluate classifiers using ROC and AUC.

ROC: Receiver Operating Characteristic; plots True Positive Rate against False Positive Rate at different thresholds.

```
TPR = TP / (TP + FN)
FPR = FP / (FP + TN)
```

AUC: Area Under the ROC Curve. AUC=1.0 is perfect separation; AUC≈0.5 is random guessing.

```
y_prob = model.predict_proba(X_test)[:, 1]
roc_curve(y_test, y_prob)
roc_auc_score(y_test, y_prob)
```

Why probabilities? ROC evaluates many decision thresholds, so it needs continuous scores/probabilities rather than only hard predictions.

Practical 11 — Cross-Validation

Aim: Check whether model performance is reliable rather than dependent on one train/test split.

5-Fold Cross-Validation

The dataset is divided into five folds. Each fold is used once for testing while the other four are used for training. The results are then averaged.

StratifiedKFold

Maintains approximately the same class proportions in each fold, which is especially useful for imbalanced fraud data.

Pipeline

```
make_pipeline(StandardScaler(), LogisticRegression(...))
```

A pipeline combines preprocessing and model training. It helps ensure scaling occurs correctly within each cross-validation training process and reduces data leakage risk.

Core ML Concepts — Must Know

Concept	Short answer
Supervised learning	Training data has labels. Examples: Decision Tree, KNN, Logistic Regression, Random Forest.
Unsupervised learning	No target labels are required. Example: Isolation Forest.
Classification	Predicts categories such as fraud/normal or malware/benign.
Training data	Used to learn model patterns.
Testing data	Used to evaluate performance on unseen data.
X	Input features.
y	Target/output label.
fit()	Trains the model.
predict()	Produces class predictions.

Evaluation Metrics — Must Know

Accuracy: (TP + TN) / (TP + TN + FP + FN)

Precision: TP / (TP + FP) — of predicted positives, how many were actually positive?

Recall: TP / (TP + FN) — of actual positives, how many were detected?

F1-score: 2PR / (P + R) — balances precision and recall.

Confusion Matrix: TP = correct positive, TN = correct negative, FP = negative predicted positive, FN = positive predicted negative.

30-Second Summary of All Practicals

Practical	Main concept	One-line explanation
1	Anaconda/Python	Set up environment and execute basic Python.
2	Decision Tree	Classify files as malware/benign.
3/4	Network IDS	Decision Tree + Isolation Forest for attacks/anomalies.
5	PCA + KNN	Facial recognition and authentication.
6	KNN	Authenticate users from typing behaviour.
7	Fraud Detection	Compare Logistic Regression and Random Forest.
8	GAN	Generate adversarial fraud samples.
9	Feature Engineering	Create stronger fraud-detection features.
10	ROC/AUC	Measure classifier discrimination across thresholds.
11	Cross-validation	Check whether model performance is reliable across splits.

If the Examiner Says: “Explain Your Practical”

“First, we import the required Python libraries and prepare the dataset. We separate the input features X from the target y. Then we split the data into training and testing sets. The selected machine-learning algorithm is trained using the training data and then used to predict unseen test data. Finally, we evaluate the model using appropriate metrics such as accuracy, precision, recall, F1-score, confusion matrix, ROC-AUC, or cross-validation depending on the practical.”

■ Highest-Priority 10 Things to Memorize

1. X = features; y = target.

2. fit() = train; predict() = predict.

3. Decision Tree = supervised classification.

4. Isolation Forest = unsupervised anomaly detection.

5. KNN = classification based on nearest points.

6. PCA = dimensionality reduction.

7. Random Forest = many Decision Trees + voting.

8. Precision/Recall/F1 are important for imbalanced fraud data.

9. ROC-AUC measures discrimination across thresholds.

10. Cross-validation checks model reliability across multiple splits.